

## Procedure

---

1. Create a simple **HTML registration form** containing the fields **Full Name**, **Email Address**, **Feedback**, and a **Register** button.
  2. Initially, do not apply any **validation attributes or security checks** to the input fields. Display the submitted values directly in the preview panel using **innerHTML**.
  3. Test the vulnerable form by entering an **invalid email address** and an XSS payload, such as an image tag with an **onerror** attribute containing a script.
  4. Click **Register** and observe that the injected HTML is rendered and the script is executed, demonstrating the **XSS vulnerability**.
  5. Create a second version of the form with **client-side validation**. Allow only letters and spaces in the name field (2–50 characters), validate the email using a suitable pattern, and make the feedback field mandatory.
  6. Display **red inline error messages** for invalid inputs and prevent the form from being submitted until all validation checks are passed.
  7. Implement **output encoding** before displaying submitted data by converting **&**, **<**, **>**, **"**, and **'** into their corresponding HTML entities.
  8. Test the secure form with invalid details and the same XSS payload. Verify that validation blocks invalid input and that the payload is displayed as **plain text without being executed**. Capture screenshots of the vulnerable and secure form results for documentation.
- 

## Source Code

---

*AnshumanDash\_VulnerableForm.html (no validation, unescaped inner HTML output)*

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>SecureNet Registration Portal – Vulnerable Version</title>
<style>
  * { box-sizing: border-box; }
  body {
    margin: 0;
    font-family: 'Segoe UI', 'Helvetica Neue', Arial, sans-serif;
    background: #eef1f6;
    color: #1f2937;
```

```
}
header {
  background: #26324a;
  color: #ffffff;
  padding: 22px 40px;
}
header h1 { margin: 0; font-size: 22px; font-weight: 600; }
header p { margin: 4px 0 0; font-size: 13px; color: #b7c0d1; }
.pill {
  display: inline-block;
  margin-top: 10px;
  padding: 4px 14px;
  border-radius: 20px;
  font-size: 12px;
  font-weight: 700;
  letter-spacing: 0.5px;
  background: #7a1f1f;
  color: #ffd9d9;
}
main {
  max-width: 620px;
  margin: 36px auto;
  padding: 0 20px;
}
.card {
  background: #ffffff;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(20,30,50,0.08);
  padding: 30px 34px;
  border: 1px solid #e3e7ee;
}
.card h2 { margin-top: 0; font-size: 17px; color: #26324a; }
label { display: block; font-size: 13px; font-weight: 600; margin: 16px
0 6px; color: #3b4557; }
input, textarea {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid #cfd6e2;
  border-radius: 6px;
  font-size: 14px;
  font-family: inherit;
}
textarea { resize: vertical; min-height: 70px; }
button {
  margin-top: 20px;
  background: #26324a;
  color: #fff;
  border: none;
  padding: 11px 22px;
  border-radius: 6px;
  font-size: 14px;
}
```

```

    font-weight: 600;
    cursor: pointer;
}
#outputPanel {
    display: none;
    margin-top: 26px;
    padding: 18px 20px;
    background: #fbfbfc;
    border: 1px dashed #c7cede;
    border-radius: 8px;
    font-size: 14px;
}
#outputPanel h3 { margin: 0 0 10px; font-size: 13px; color: #667085;
text-transform: uppercase; letter-spacing: 0.5px; }
.note {
    margin-top: 10px;
    font-size: 12px;
    color: #8a93a3;
}
footer {
    text-align: center;
    font-size: 11px;
    color: #97a0b3;
    margin: 30px 0 10px;
}
</style>
</head>
<body>

<header>
    <h1>SecureNet Registration Portal</h1>
    <p>Cyber Security Lab – Experiment 5</p>
    <div class="pill">VULNERABLE VERSION – NO VALIDATION / NO
ENCODING</div>
</header>

<main>
    <div class="card">
        <h2>Create an account</h2>
        <form id="regForm">
            <label for="fullName">Full Name</label>
            <input type="text" id="fullName" name="fullName">

            <label for="email">Email Address</label>
            <input type="text" id="email" name="email">

            <label for="feedback">Feedback / Comments</label>
            <textarea id="feedback" name="feedback"></textarea>

            <button type="submit">Register</button>
        </form>

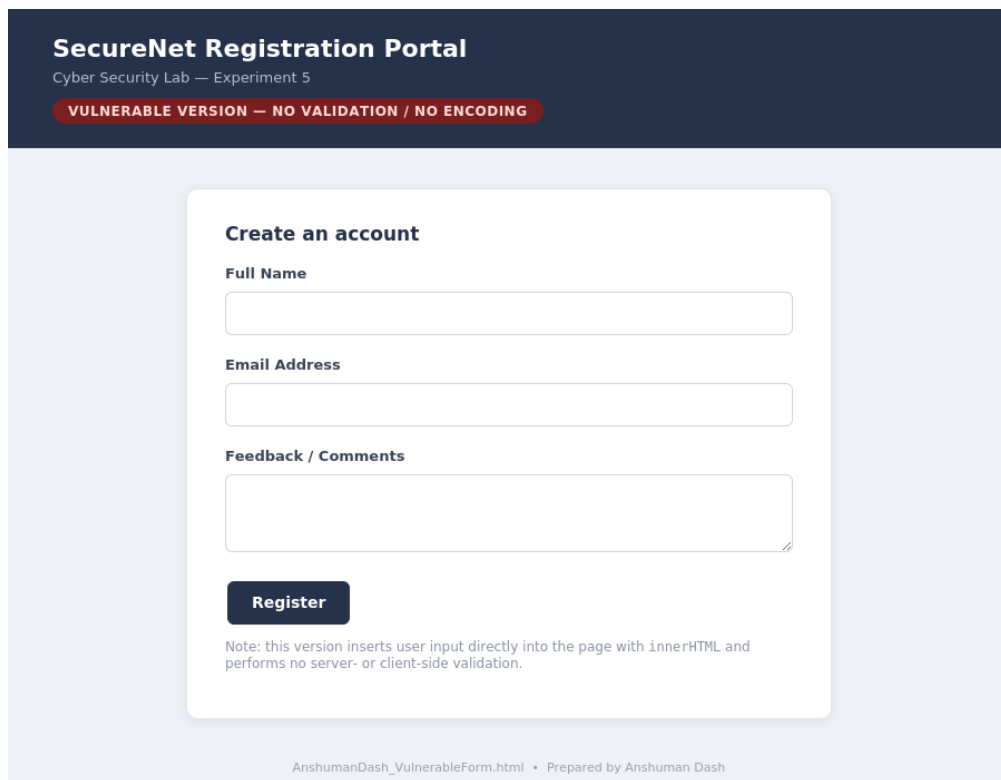
```



```
    });  
  }  
</script>  
  
</body>  
</html>
```

## Output

Fig. 1 — Vulnerable form in its initial, empty state.



The screenshot displays the 'SecureNet Registration Portal' interface. At the top, a dark blue header contains the title 'SecureNet Registration Portal', the subtitle 'Cyber Security Lab — Experiment 5', and a red warning banner that reads 'VULNERABLE VERSION — NO VALIDATION / NO ENCODING'. The main content area features a white registration form titled 'Create an account'. The form includes three input fields: 'Full Name', 'Email Address', and 'Feedback / Comments'. Below these fields is a dark blue 'Register' button. A note at the bottom of the form states: 'Note: this version inserts user input directly into the page with innerHTML and performs no server- or client-side validation.' The footer of the page indicates the file name 'AnshumanDash\_VulnerableForm.html' and credits 'Prepared by Anshuman Dash'.

*Fig. 1: Vulnerable registration form before any data is entered.*

Fig. 2 — Vulnerable form after submitting an invalid email together with a script-carrying feedback payload; the injected markup is executed and a warning banner generated entirely by the attacker payload appears in the preview panel.

# SecureNet Registration Portal

Cyber Security Lab — Experiment 5

**VULNERABLE VERSION — NO VALIDATION / NO ENCODING**

## Create an account

Full Name

Anshuman Dash

Email Address

anshumandash2006pro@gmail.com

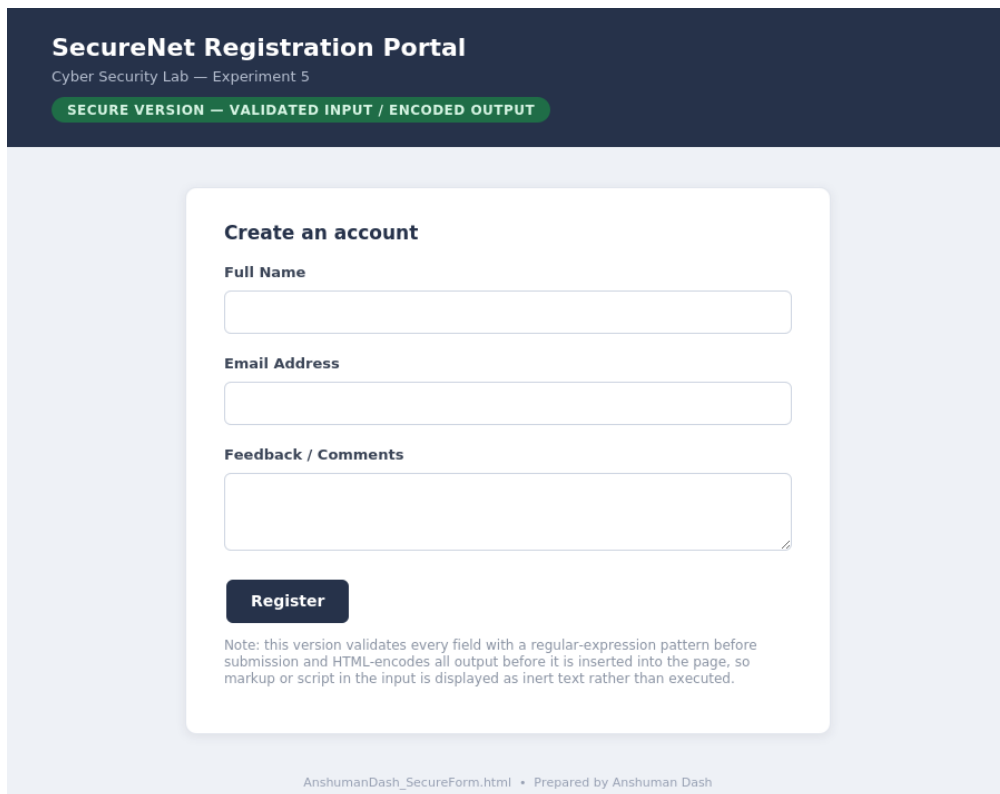
Feedback / Comments

**Register**

Note: this version inserts user input directly into the page with `innerHTML` and performs no server-side validation.

*Fig. 2: Injected payload executes — no validation, no encoding.*

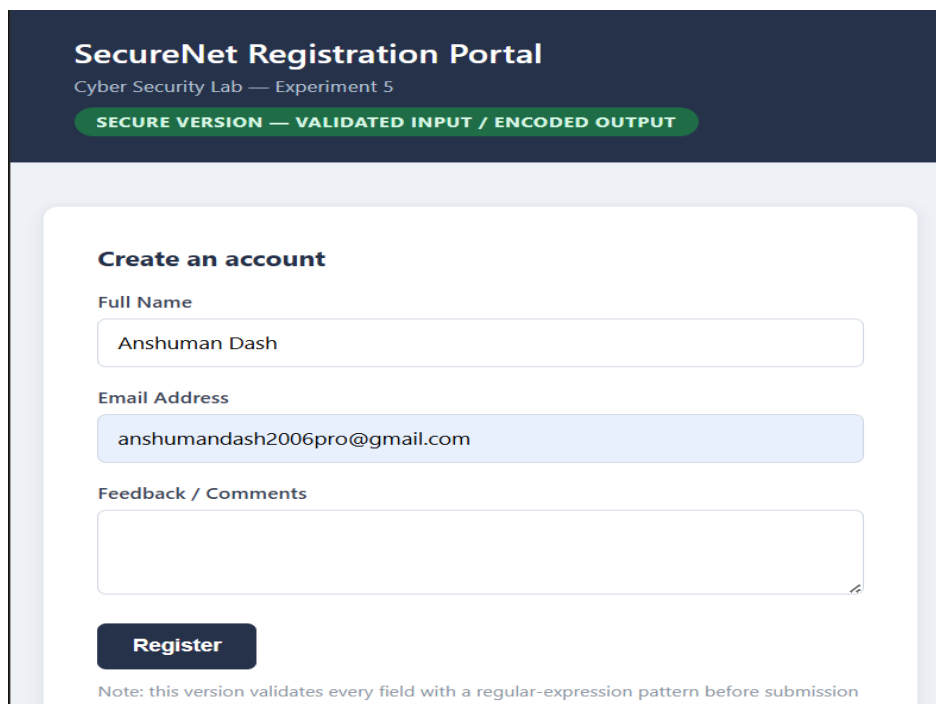
Fig. 3 — Secure form in its initial, empty state, visually identical to the vulnerable version.



The image shows a web form titled "SecureNet Registration Portal" with a subtitle "Cyber Security Lab — Experiment 5". A green banner below the header reads "SECURE VERSION — VALIDATED INPUT / ENCODED OUTPUT". The form itself is titled "Create an account" and contains three input fields: "Full Name", "Email Address", and "Feedback / Comments". Below these fields is a dark blue "Register" button. A note at the bottom of the form states: "Note: this version validates every field with a regular-expression pattern before submission and HTML-encodes all output before it is inserted into the page, so markup or script in the input is displayed as inert text rather than executed." At the very bottom of the page, small text reads "AnshumanDash\_SecureForm.html • Prepared by Anshuman Dash".

*Fig. 3: Secure registration form before any data is entered.*

Fig. 4 — Secure form after submitting an invalid name and email with an empty feedback field; regex-based validation rejects the input and inline error messages are shown, and the submission is blocked.



The image shows the same "SecureNet Registration Portal" form, but now with data entered. The "Full Name" field contains "Anshuman Dash" and the "Email Address" field contains "anshumandash2006pro@gmail.com". The "Feedback / Comments" field is empty. The "Register" button is still present. The same note about validation and HTML encoding is at the bottom. The form is styled with a light blue background and a dark blue header.

*Fig. 4: Client-side validation rejects malformed input.*